

스트리밍 알고리즘 2종 구현 및 정확도·메모리 트레이드오프 분석

1. 데이터셋 설명

본 실험에서는 **MovieLens 1M** 데이터셋을 사용하였다. 해당 데이터셋의 평점 기록을 사용자 이벤트 스트림으로 간주하였다. 각 레코드는 `UserID::MovieID::Rating::Timestamp` 형식으로 구성되어 있으며, 본 실험에서는 전체 데이터를 한 번에 메모리에 올리지 않고 한 줄씩 읽는 스트리밍 방식으로 처리하였다.

[GIT 저장소 링크](#)

항목	내용
데이터셋	MovieLens 1M
스트림 단위	사용자-영화 평점 이벤트
전체 레코드 수	1,000,209
고유 이벤트 수	1,000,209
고유 영화 수	3,706
파일 형식	DAT 텍스트 파일

2. 선택한 알고리즘 개요

2.1 Bloom Filter

Bloom Filter는 원소가 집합에 포함되어 있는지를 근사적으로 판정하는 확률적 자료구조이다. 여러 개의 hash 함수를 이용해 bit array의 위치를 1로 설정한다. 조회 시 해당 위치가 모두 1이면 포함되어 있다고 판단한다. 이때 실제로는 없는 원소를 있다고 판단하는 **false positive**가 발생할 수 있지만, 이미 삽입된 원소를 없다고 판단하는 **false negative**는 발생하지 않는다.

2.2 Count-Min Sketch

Count-Min Sketch는 데이터 스트림에서 항목별 빈도를 근사 추정하기 위한 자료구조이다. 여러 개의 hash 함수와 2차원 count table을 사용하며, 항목의 빈도 추정값은 각 row에서 얻은 count 값 중 최솟값으로 계산한다. hash 충돌로 인해 실제 빈도보다 크게 추정될 수 있지만, 일반적으로 메모리를 크게 줄이면서 빈도 정보를 근사적으로 유지할 수 있다.

3. 구현 방식

두 알고리즘 모두 Python으로 직접 구현하였다. Bloom Filter는 `bytearray` 기반 bit array를 사용하였고, Count-Min Sketch는 `numpy` 배열 기반의 2차원 count table을 사용하였다. hash 함수는 `hashlib.blake2b`를 이용해 deterministic hash를 생성하였다.

알고리즘	구현 방식	Ground Truth
Bloom Filter	bytearray 기반 bit array, double hashing	Python set 기반 실제 포함 여부
Count-Min Sketch	numpy 2차원 count table, double hashing	Python dictionary 기반 정확한 movie_id별 빈도

소스코드 GitHub 주소: <https://github.com/your-id/streaming-algorithms-report>

4. 실험 환경

항목	값
Python 버전	3.12.13
Platform	Linux-6.6.122-x86_64-with-glibc2.35
CPU Core 수	2
RAM	12.67 GB

5. 파라미터 설정

5.1 Bloom Filter

Bloom Filter는 `bits_per_item`을 변경하며 실험하였다. 전체 bit array 크기는 `전체 레코드 수 × bits_per_item`으로 설정하였다. hash 함수 개수는 일반적인 최적값인 $k = (m / n) \ln 2$ 에 근접하도록 설정하였다.

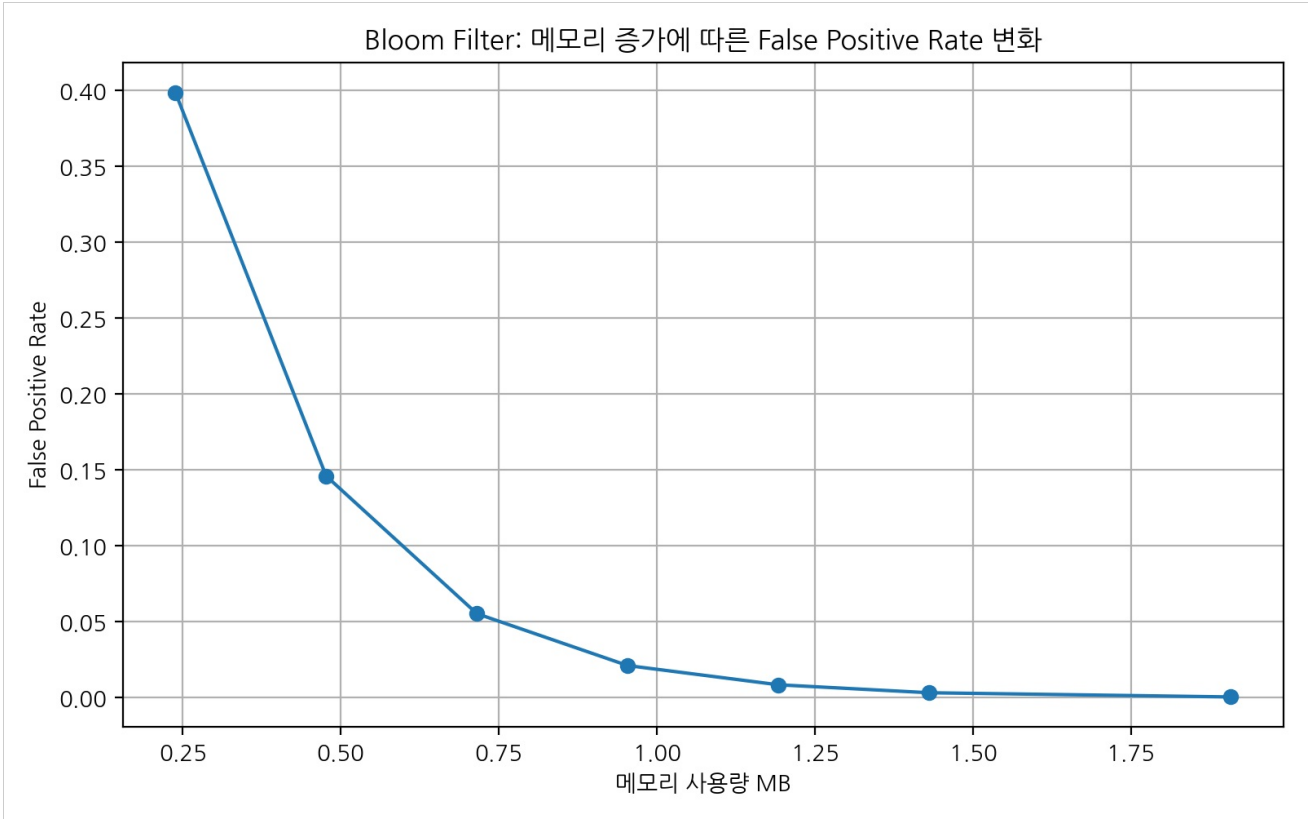
5.2 Count-Min Sketch

Count-Min Sketch는 `width`와 `depth`를 변경하며 실험하였다. width가 증가하면 hash 충돌 가능성이 낮아지고, depth가 증가하면 여러 row 중 최솟값을 사용하기 때문에 과대 추정 오차를 줄일 수 있다.

6. 정확도 비교 결과

6.1 Bloom Filter 결과

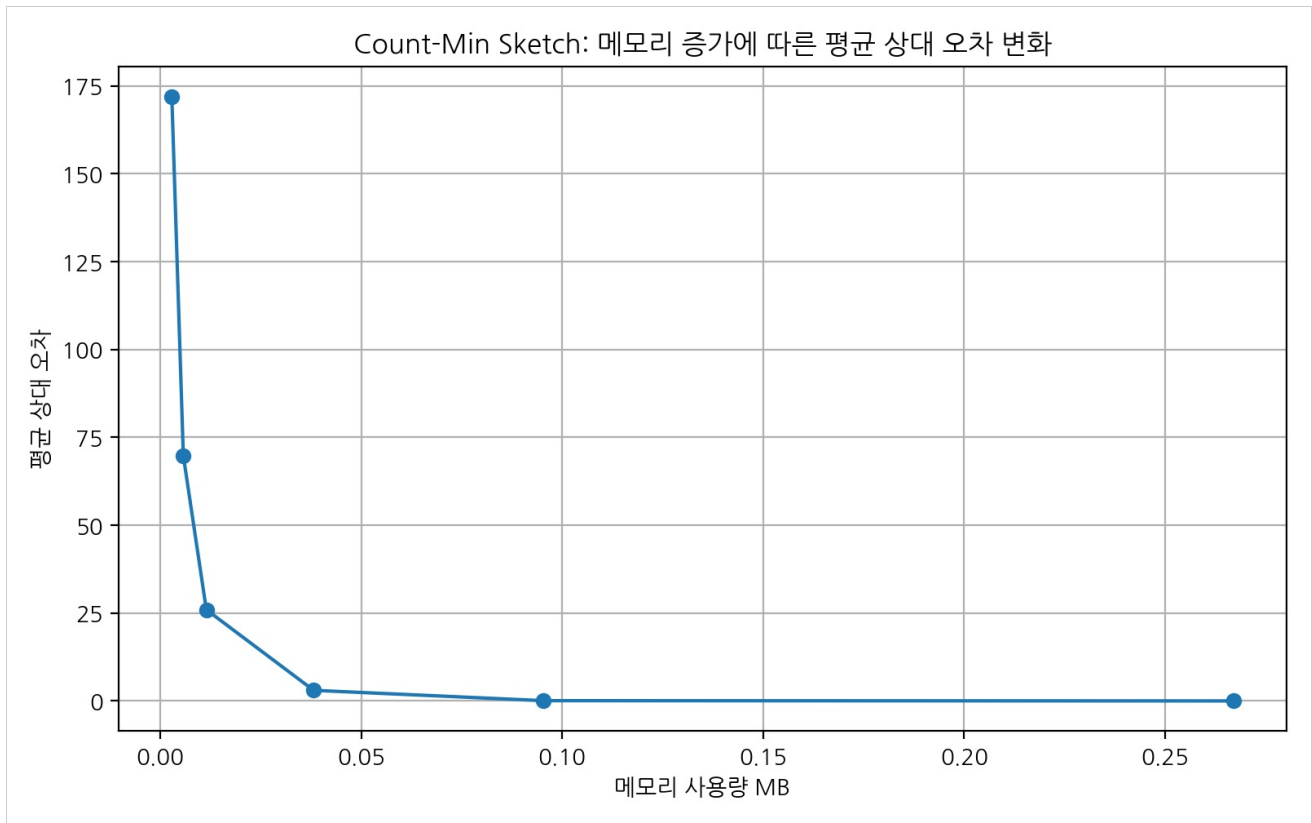
algorithm	bits_per_item	m_bits	k_hashes	memory_bytes	memory_mb	build_time_sec	throughput_records_per_sec	true_positive_rate	false_positive_rate	theoretical_fpr
Bloom Filter	2	2000418	1	250053	0.238469	3.388396	295186.544525	1.000000	0.398600	0.393469
Bloom Filter	4	4000836	3	500105	0.476937	4.519950	221287.648510	1.000000	0.145960	0.146892
Bloom Filter	6	6001254	4	750157	0.715405	4.184796	239010.220412	1.000000	0.055260	0.056057
Bloom Filter	8	8001672	6	1000209	0.953874	5.401523	185171.652377	1.000000	0.021080	0.021577
Bloom Filter	10	10002090	7	1250262	1.192343	5.016333	199390.483091	1.000000	0.008420	0.008194
Bloom Filter	12	12002508	8	1500314	1.430811	6.022317	166083.759815	1.000000	0.003180	0.003142
Bloom Filter	16	16003344	11	2000418	1.907747	6.366276	157110.537487	1.000000	0.000420	0.000459



Bloom Filter는 메모리 사용량이 증가할수록 False Positive Rate가 전반적으로 감소하였다. 가장 낮은 False Positive Rate는 **bits_per_item=16** 설정에서 관찰되었으며, 실제 FPR은 **0.000420** 이었다.

6.2 Count-Min Sketch 결과

algorithm	width	depth	memory_bytes	memory_mb	build_time_sec	throughput_records_per_sec	mean_absolute_error	median_absolute_error	max_absolute_error	mean_relative_error	median_relative_error	max_relative_error	overestimate_rate
Count-Min Sketch	250	3	3000	0.002861	5.129613	194987.231557	2531.521856	2472.000000	7253	171.976374	18.745130	6014.000000	1.000000
Count-Min Sketch	500	3	6000	0.005722	4.437708	225388.668055	1014.125202	933.500000	4787	69.769508	6.762632	2789.000000	1.000000
Count-Min Sketch	1000	3	12000	0.011444	4.630763	215992.258176	356.837561	250.500000	2332	25.961499	1.760299	2034.000000	1.000000
Count-Min Sketch	2000	5	40000	0.038147	5.918804	168988.373639	38.469239	0.000000	1367	3.018289	0.000000	522.000000	1.000000
Count-Min Sketch	5000	5	100000	0.095367	5.345275	187120.217532	1.514031	0.000000	516	0.075209	0.000000	51.500000	1.000000
Count-Min Sketch	10000	7	280000	0.267029	6.943845	144042.528880	0.005666	0.000000	11	0.001359	0.000000	5.000000	1.000000

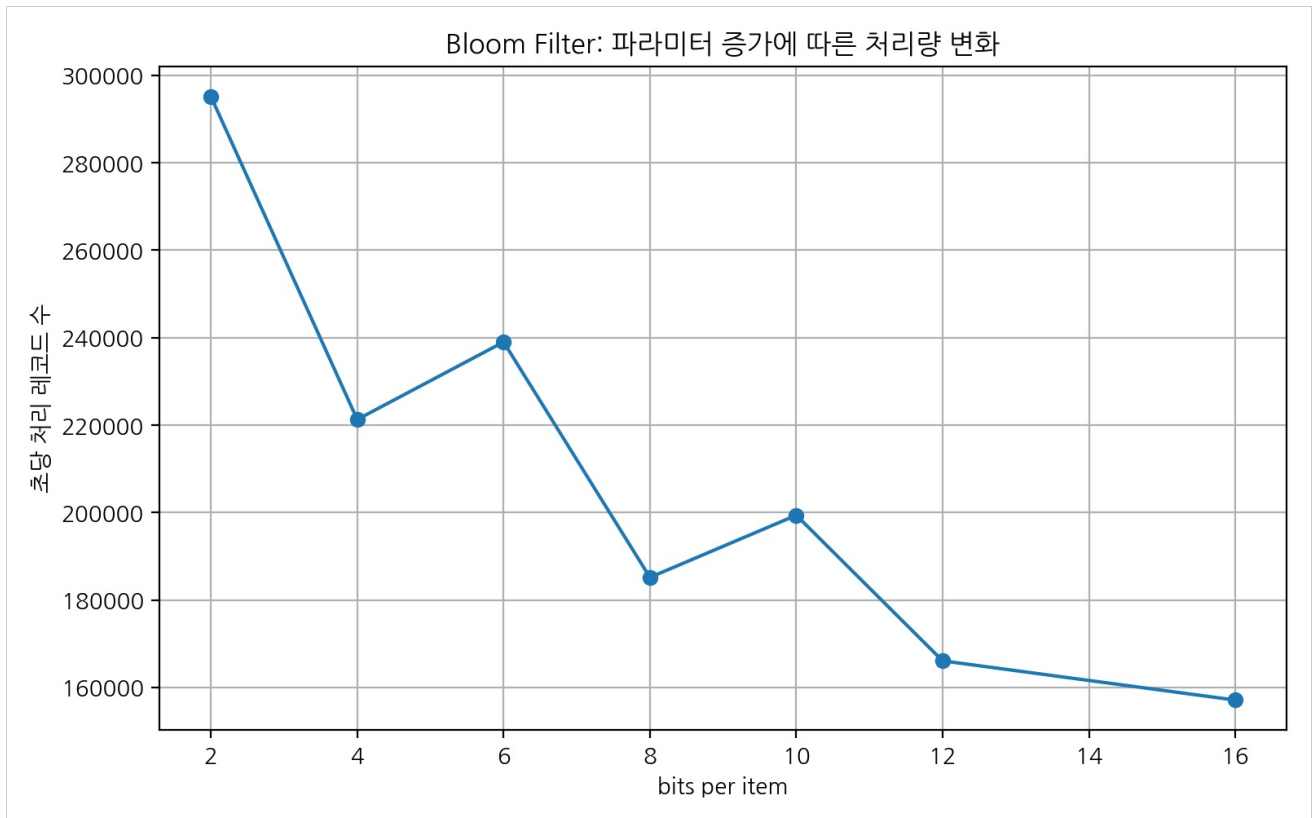


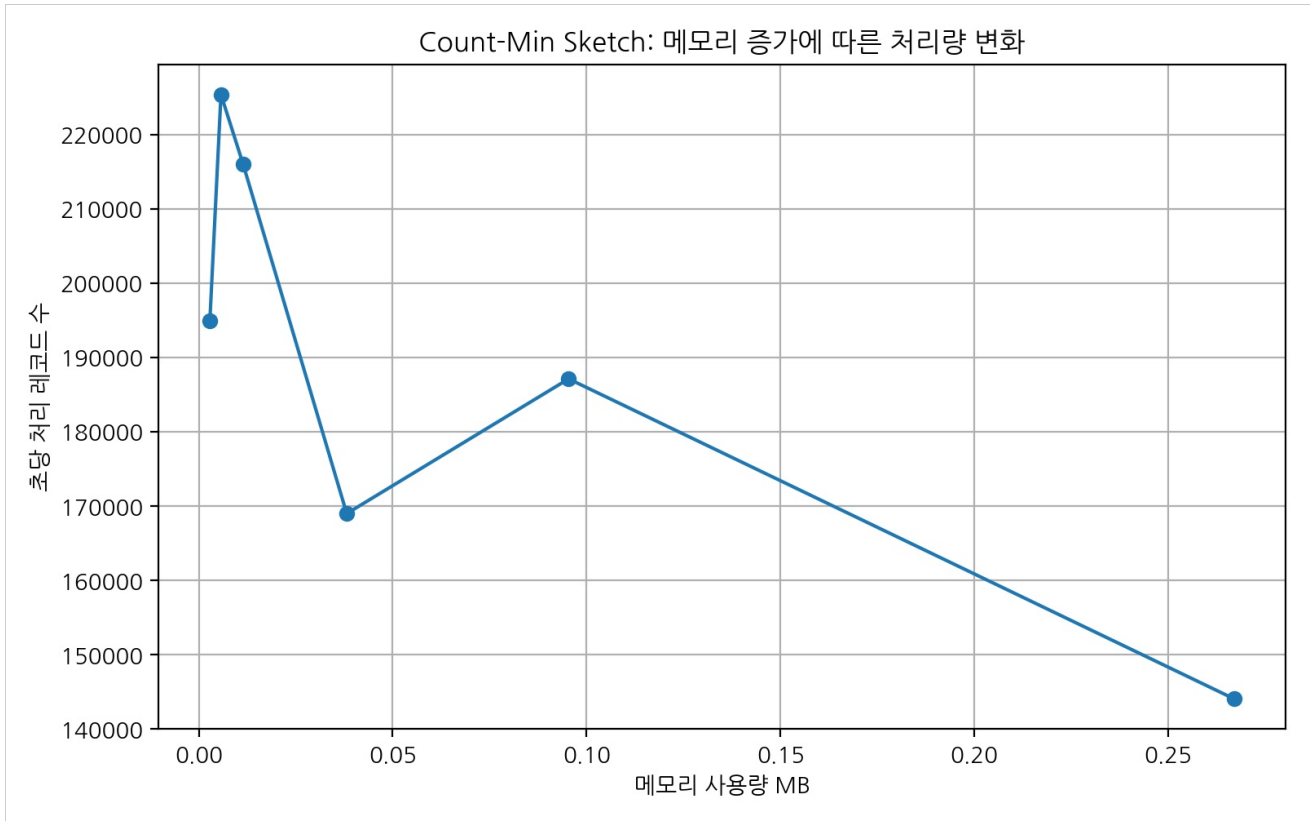
Count-Min Sketch는 width와 depth가 증가할수록 평균 상대 오차가 전반적으로 감소하였다. 가장 낮은 평균 상대 오차는 **width=10000, depth=7** 설정에서 관찰되었으며, 평균 상대 오차는 **0.001359** 이었다.

7. 메모리 사용량 비교 결과

Bloom Filter의 메모리 사용량은 bit array 크기에 의해 결정되며, bits per item이 커질수록 선형적으로 증가하였다. Count-Min Sketch의 메모리 사용량은 $\text{width} \times \text{depth} \times \text{counter}$ 크기에 비례한다. 따라서 두 알고리즘 모두 정확도 향상을 위해서는 더 큰 메모리 사용량이 필요했다.

8. 처리 시간 비교 결과





처리 시간은 hash 계산 횟수와 자료구조 업데이트 횟수에 영향을 받았다. Bloom Filter는 hash 함수 개수가 증가할수록 삽입과 조회 비용이 증가할 수 있다. Count-Min Sketch는 depth가 증가할수록 각 이벤트마다 업데이트해야 하는 row 수가 증가하므로 처리량이 감소할 수 있다.

9. 알고리즘별 장단점 분석

알고리즘	장점	단점	적합한 상황
Bloom Filter	매우 적은 메모리로 membership test 가능, false negative 없음	false positive 발생, 삭제 어려움, 빈도 추정 불가	중복 검사, 이미 본 사용자/URL/이벤트 확인
Count-Min Sketch	대용량 스트림에서 항목별 빈도 추정 가능, 메모리 효율적	hash 충돌로 인한 과대 추정 가능, 정확한 값은 아님	인기 상품, 인기 URL, hot key, 이벤트 빈도 분석

10. 최종 분석 질문

Q1. 정확도와 메모리 사이에는 어떤 trade-off가 있었는가?

Bloom Filter에서는 bit array 크기가 커질수록 false positive가 감소했지만 메모리 사용량이 증가하였다. Count-Min Sketch에서는 width와 depth가 커질수록 hash 충돌이 줄어들 빈도 추정 오차가 감소했지만, 그만큼 count table 크기가 커져 메모리 사용량이 증가하였다. 즉, 두 알고리즘 모두 정확도를 높이려면 더 많은 메모리를 사용해야 하는 trade-off가 있었다.

Q2. 파라미터 증가가 항상 성능 향상으로 이어졌는가?

항상 그렇지 않았다. 파라미터 증가는 정확도 측면에서는 대체로 유리했지만, 처리 시간 측면에서는 불리할 수 있었다. Bloom Filter에서 hash 함수 개수가 많아지면 삽입과 조회 시 hash 계산 비용이 증가한다. Count-Min Sketch에서도 depth가 커지면 각 레코드마다 여러 row를 업데이트해야 하므로 처리량이 감소할 수 있다. 따라서 파라미터 증가는 정확도 개선과 시간 비용 증가 사이의 균형으로 해석해야 한다.

Q3. 어떤 알고리즘이 가장 실용적이라고 판단되는가?

본 실험에서는 Count-Min Sketch가 더 실용적이라고 판단된다. 실제 로그 분석에서는 특정 이벤트, URL, 상품, 사용자 행동의 발생 빈도를 파악하는 경우가 많기 때문이다. Count-Min Sketch는 전체 데이터를 저장하지 않고도 항목별 빈도를 근사 추정할 수 있어, 대용량 서비스 로그 분석에 적합하다.

Q4. 실제 서비스 로그 분석에 적용한다면 어떤 알고리즘을 선택할 것인가?

실제 서비스 로그 분석에서는 Count-Min Sketch를 선택할 것이다. 예를 들어 인기 페이지, 많이 클릭된 상품, 비정상적으로 많이 발생하는 에러 코드, hot key 탐지 등은 모두 빈도 추정 문제로 볼 수 있다. Count-Min Sketch는 제한된 메모리 안에서 이러한 빈도 정보를 빠르게 추정할 수 있으므로 실시간 로그 모니터링이나 대시보드 시스템에 적용하기 적합하다. 다만 중복 이벤트 제거 또는 이미 처리된 요청 여부 확인이 목적이라면 Bloom Filter를 함께 사용할 수 있다.

11. 결론

본 과제에서는 MovieLens 1M 데이터셋을 스트리밍 방식으로 처리하며 Bloom Filter와 Count-Min Sketch를 직접 구현하고 비교하였다. Bloom Filter는 membership test에 매우 효율적이며, Count-Min Sketch는 빈도 추정에 적합하였다. 실험 결과, 두 알고리즘 모두 메모리를 더 많이 사용할수록 정확도는 개선되었지만, 파라미터 증가로 인해 처리 시간이 늘어날 수 있음을 확인하였다. 따라서 스트리밍 알고리즘을 실제 서비스에 적용할 때는 정확도, 메모리, 처리량 사이의 균형을 고려해 파라미터를 설정해야 한다.

